ĐỒ ÁN P1 · BIG DATA ANALYTICS AND MANAGEMENT

So sánh Delta Lake, Iceberg và Hudi dưới workload cập nhật nhiều cho dữ liệu khí tượng – thủy văn



Update-Heavy Workloads on Small Clusters: An Experimental Comparison of Delta Lake, Apache Iceberg and Apache Hudi for IoT and Hydro-Meteorological Data

Hà Văn Vinh · Đỗ Thị Lan

Học viên cao học · Môn Big Data Analytics and Management (TLU 2026)

Tóm tắt

Có dữ liệu trễ hay không mới quyết định chi phí; Iceberg Merge-on-Read rẻ nhất

237 / 237

lần chạy trên Kaggle khớp ground truth ở mọi checkpoint: 3 format × 2 chế độ ghi × 4 mức u × 3 chu kỳ compaction, lặp 3 lần

59 s

tổng chi phí của Iceberg MoR ở u = 20%: rẻ nhất ở mọi u > 0, rẻ hơn 2,6–4,3 lần so với mọi cấu hình Delta và Hudi (RQ1)

1,4–3,8×

chi phí tăng ngay khi có dữ liệu trễ, rồi chỉ chênh tối đa 8% từ u = 1% đến u = 20% (RQ1)

mỗi 20 batch

compaction đủ đưa độ trễ đọc MoR về ngang CoW với 8–17 s; compaction mỗi 5 batch chỉ tốn thêm (RQ2)

Một node 4 vCPU, bảng ≈ 222 MB; thời gian giữa các mức u đã chuẩn hóa theo tốc độ từng VM.

Bối cảnh

Dữ liệu giám sát là workload cập nhật nhiều, không phải đọc nhiều như TPC-DS

Append liên tục

Số đo mưa, mực nước, chất lượng nước đến mỗi 10 giây đến vài phút, nạp theo micro-batch.

Dữ liệu đến trễ

Số đo trễ hoặc được hiệu chỉnh vài giờ sau phải ghi đè giá trị cũ (upsert).

Xóa theo trạm

Trạm hỏng: xóa dữ liệu của trạm đó trong cả ngày.

3 TB — LHBench (CIDR 2023), LST-Bench (SIGMOD 2024): TPC-DS trên cụm đám mây lớn

1 node — Cơ quan, trường ở Việt Nam thường có 1–4 máy chủ: đề tài chạy 4 vCPU, ≈ 222 MB

Câu hỏi nghiên cứu và đóng góp

Ba câu hỏi, ba giả thuyết rút từ LHBench và LST-Bench, kiểm chứng trên 228 lần chạy

Câu hỏi nghiên cứu

RQ 1 Với u ∈ {0, 1, 5, 20}%, format và chế độ ghi nào có tổng chi phí thấp nhất?

RQ 2 Chu kỳ compaction nào cân bằng số file nhỏ và độ trễ đọc?

RQ 3 Metadata tăng thế nào theo số commit, ảnh hưởng gì tới time travel?

Đóng góp

C1 · Bộ sinh workload **sensor-LST** mã nguồn mở, tất định theo seed, có ground truth

C2 · So sánh có kiểm soát, tái lập được: 3 format + Parquet, 2 chế độ, 3 chu kỳ compaction

C3 · Giải thích bằng thiết kế format, danh sách "bẫy" và bảng khuyến nghị

C4 · Demo trực tiếp trên localhost

Công trình liên quan

Các đánh giá hiện có dùng TPC-DS trên cụm lớn; chưa có workload cảm biến cập nhật nhiều

Lakehouse và table format

Armbrust và cs. (CIDR 2021) đề xuất Lakehouse; Delta Lake (PVLDB 2020), đặc tả Iceberg, Hudi: dữ liệu Parquet bất biến + nhật ký phiên bản riêng.

Benchmark Lakehouse

LHBench (CIDR 2023): TPC-DS 3 TB, khác biệt nằm ở đường đọc; LST-Bench (SIGMOD 2024): mẫu bảo trì, time travel; AutoComp (SIGMOD 2025): file nhỏ là vấn đề vận hành.

Dữ liệu chuỗi thời gian

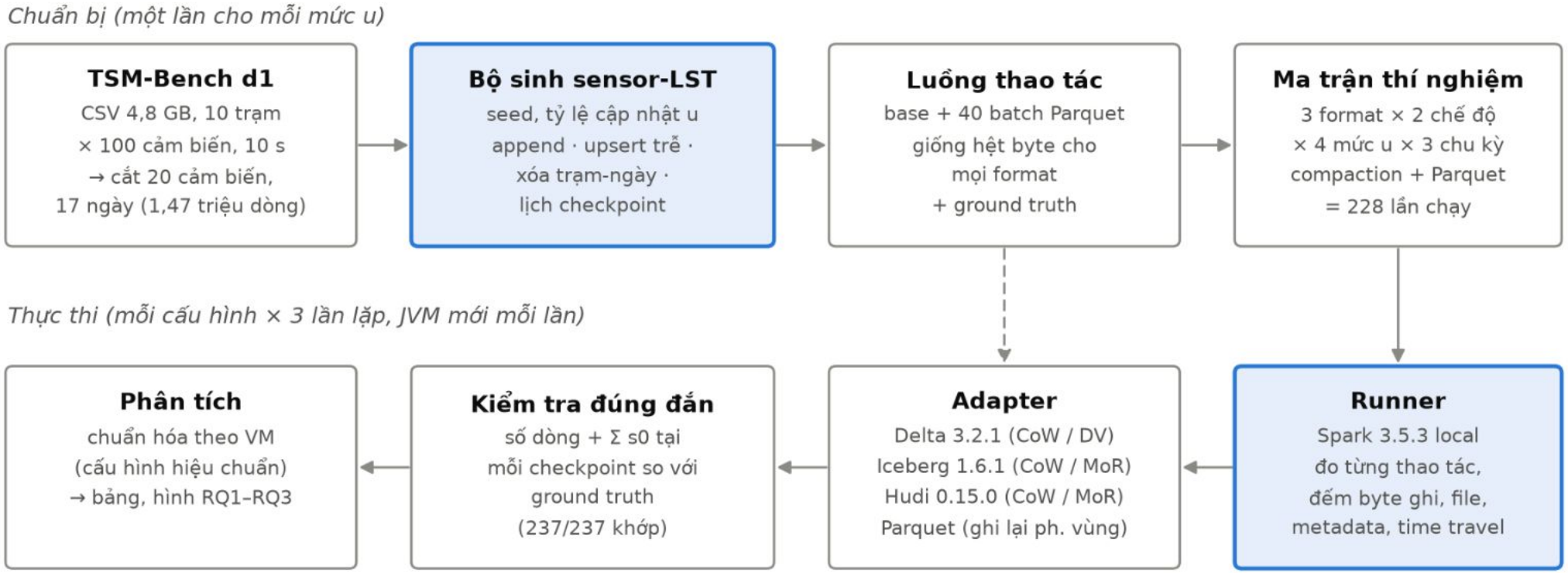
TSM-Bench (PVLDB 2023): 7 TSDB, truy vấn từ bài toán giám sát nước, dữ liệu thủy văn Thụy Sĩ. Đề tài dùng lại dữ liệu d1 và thêm hiệu chỉnh trễ, xóa.

Phương pháp benchmark trên cloud

Leitner & Cito (TOIT 2016), Uta và cs. (NSDI 2020): hiệu năng máy dùng chung biến động; cần lặp, xen kẽ ngẫu nhiên và hiệu chuẩn.

Kiến trúc hệ thống thí nghiệm

Sinh luồng thao tác một lần, phát lại trên từng layout trong JVM mới, kiểm tra với ground truth



Ba table format, hai chế độ ghi

Cùng lưu Parquet bất biến; khác nhau ở nhật ký phiên bản và cách ghi thay đổi cấp dòng

	Delta Lake 3.2.1	Iceberg 1.6.1	Hudi 0.15.0
Nhật ký phiên bản	JSON + checkpoint mỗi 10 commit	metadata JSON → manifest	timeline + metadata table
Merge-on-Read	deletion vector (bitmap)	position-delete file	log file trong file group
Khóa bản ghi	không (MERGE theo vị từ)	không (MERGE theo vị từ)	bắt buộc, có chỉ mục khóa
Compaction	REORG + OPTIMIZE	rewrite_data_files	compaction / clustering
Time travel mặc định	đến khi VACUUM	đến khi expire snapshot	≈ 23 instant gần nhất

CoW · ghi lại cả file chứa dòng bị đổi

MoR · ghi thay đổi rẻ, hợp nhất khi đọc

Compaction · gộp file nhỏ, áp dụng delete

Workload sensor-LST

Mọi format nhận cùng một luồng thao tác, giống hệt byte, kèm ground truth

1 Nạp lịch sử

7 ngày, 604.800 dòng, nạp một lần

2 40 micro-batch

mỗi batch 6 giờ: 21.600 dòng mới + u × 21.600 hiệu chỉnh trễ trong 72 giờ trước

3 Xóa trạm-ngày

mỗi 10 batch xóa một trạm trong một ngày

4 Checkpoint

mỗi 10 batch: Q1–Q5, time travel, thống kê, so với ground truth

1,47 triệu dòng

TSM-Bench d1 (PVLDB 2023): 10 trạm × 20 cảm biến × 17 ngày, mẫu 10 giây, dữ liệu thủy văn Thụy Sĩ

Q1 trung bình theo giờ, 24 giờ gần nhất · **Q2** top-5 trạm, 72 giờ
Q3 join bảng trạm, toàn bộ lịch sử · **Q4** một cảm biến, 2 giờ
Q5 count và Σ s0 toàn bảng (đồng thời là checksum)

Thiết lập thí nghiệm

228 lần chạy trên Kaggle; hai trong bốn VM chậm hơn khoảng 1,5 lần nên phải hiệu chuẩn

228 lần chạy

3 format × 2 chế độ × 4 mức u × 3 chu kỳ compaction = 72 cấu hình, cộng 4 Parquet; mỗi cấu hình 3 lần, thứ tự xáo ngẫu nhiên

4 vCPU · 31 GB

Kaggle CPU, Spark 3.5.3 local, Delta 3.2.1, Iceberg 1.6.1, Hudi 0.15.0; mỗi mức u một VM, 2–6 giờ

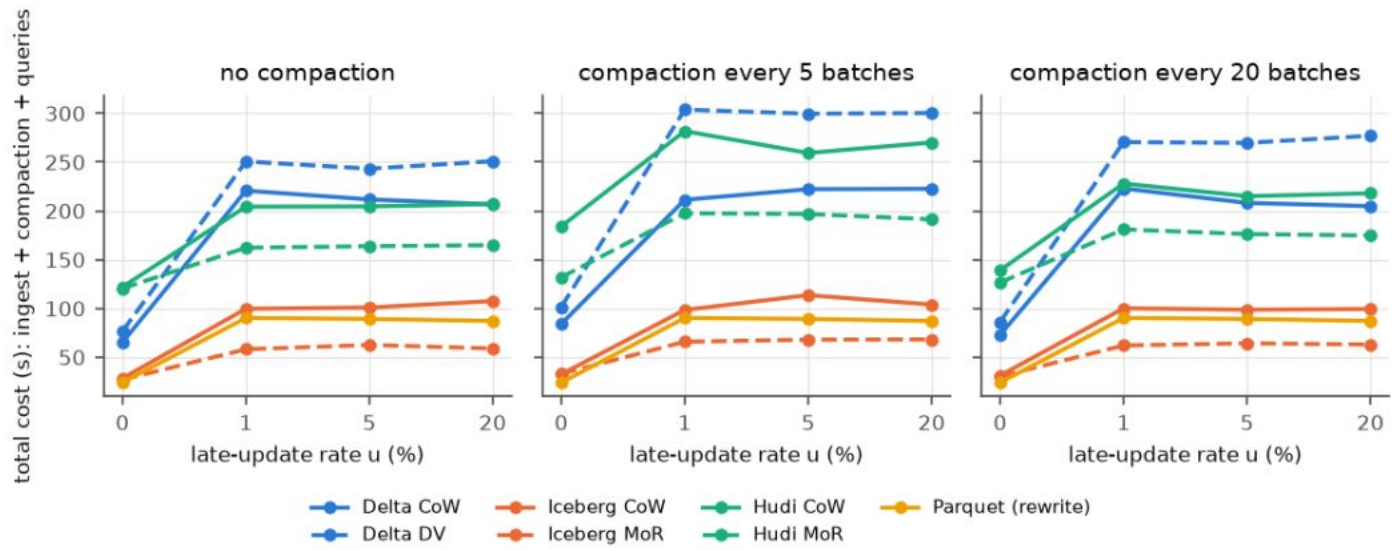
Cùng cấu hình hiệu chuẩn (Delta CoW, u = 0) trên 4 VM

VM của shard	Thời gian	Hệ số
u = 0%	65 s	1,00
u = 1%	103 s	1,58
u = 5%	70 s	1,07
u = 20%	98 s	1,50

So sánh format luôn trong cùng VM; so sánh giữa các mức u chia thời gian cho hệ số của VM.

Kết quả tổng hợp

Iceberg MoR rẻ nhất ở mọi u > 0; chi phí nhảy ở u = 1% rồi đứng yên tới 20%



Bước nhảy khi có dữ liệu trễ: CoW ×3,3–3,8; Iceberg MoR ×2,2; Hudi ×1,4–1,7

Từ u = 1% đến 20%: chênh tối đa 8% ở mọi layout

u = 20%: Iceberg MoR 59 s · Iceberg CoW 108 s · Hudi MoR 165 s · Delta DV 251 s

RQ1: Vì sao chi phí đứng yên từ 1% đến 20%?

Ở u = 1%, bản ghi hiệu chỉnh đã chạm hầu hết file của bốn phân vùng ngày gần nhất

216

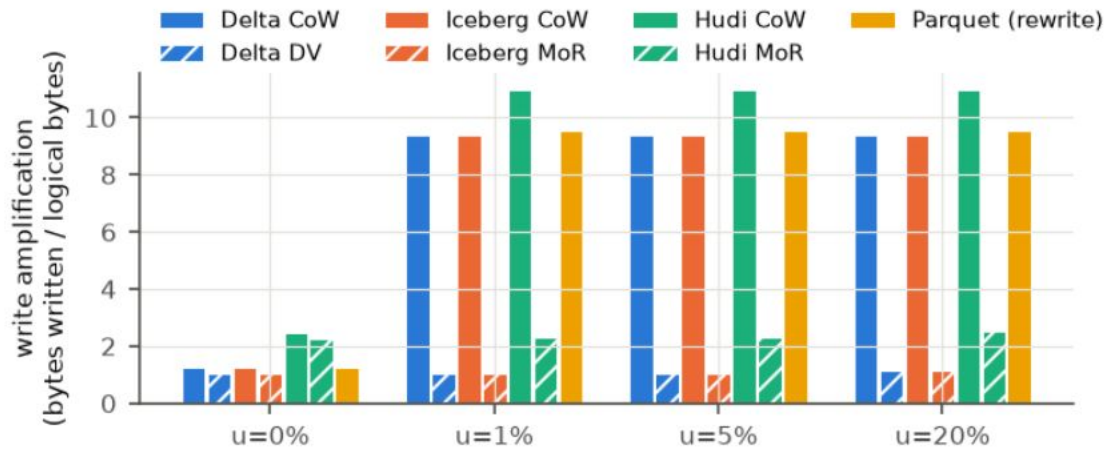
bản ghi trễ mỗi batch ở u = 1%, rải trên 72 giờ

9,4×

write amplification của mọi layout CoW khi u > 0, không đổi theo u

1,0–1,2

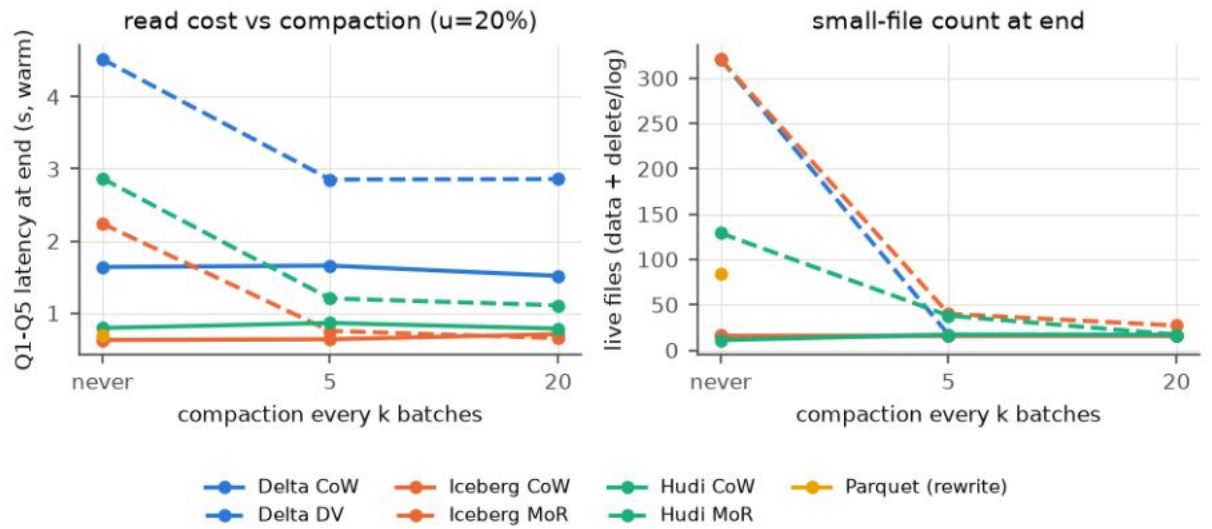
write amplification của Delta DV và Iceberg MoR



Byte không phải là thời gian: Delta DV ghi ít hơn Delta CoW 8 lần về byte nhưng nạp chậm hơn 9–17%; Iceberg MoR nạp nhanh gấp đôi Iceberg CoW.

RQ2: Compaction mỗi 20 batch là đủ

Không compaction, MoR đọc chậm 3–4 lần; compaction mỗi 5 batch chỉ tốn thêm



Không compaction: MoR giữ tới 167 + 154 file; đọc 2,24 s (Iceberg) · 2,87 s (Hudi) · 4,52 s (Delta DV)

Mỗi 20 batch: Iceberg MoR 0,65 s (CoW 0,63 s), Hudi MoR 1,11 s; chỉ tốn 8–17 s

Mỗi 5 batch: không đọc nhanh hơn, compaction tốn gấp đôi

Điểm gãy (H3): độ trễ đọc của Hudi MoR nhảy lên giữa batch 20 và 30. H2 đúng với Iceberg và Hudi, không đúng với Delta.

RQ2: Truy vấn nào chịu thiệt?

MoR chỉ làm chậm truy vấn quét toàn lịch sử; dashboard trên dữ liệu gần đây gần như không đổi

Độ trễ warm ở checkpoint cuối, u = 20% (giây)

Layout (compaction)	Q1 · 24 h	Q3 · join	Q5 · scan
Iceberg CoW (không)	0,09	0,28	0,10
Iceberg MoR (không)	0,14	1,07	0,75
Iceberg MoR (mỗi 20)	0,12	0,26	0,08
Hudi MoR (không)	0,15	1,28	0,95
Delta CoW (không)	0,28	0,54	0,33
Delta DV (không)	0,87	1,46	0,82
Delta DV (mỗi 20)	0,54	0,85	0,51

×7,8–9,7

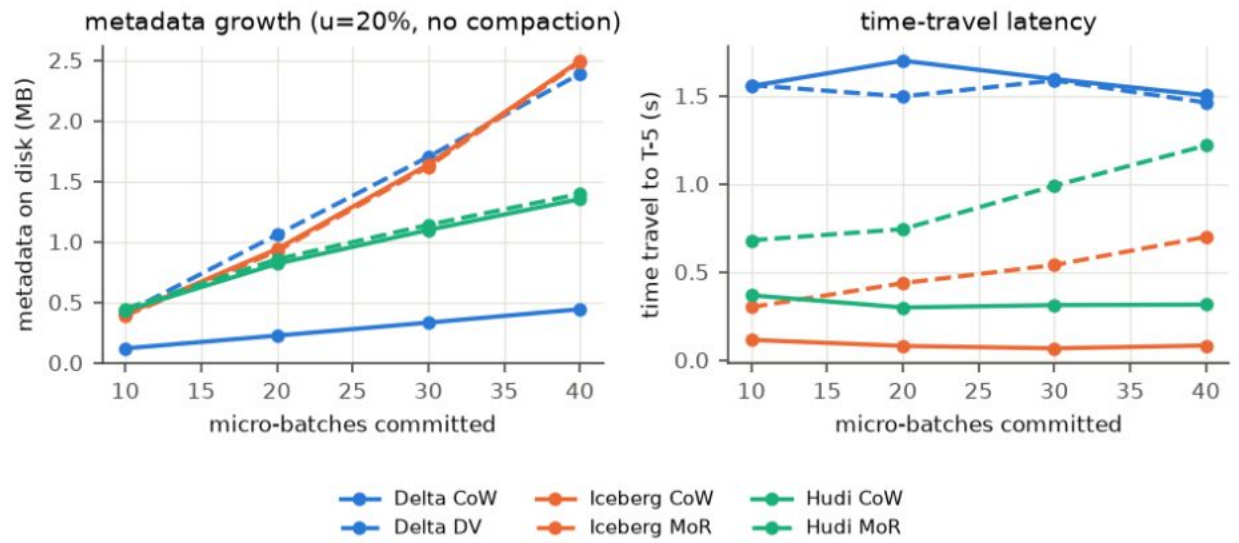
Q5 (quét toàn bảng) của Iceberg và Hudi MoR so với CoW; Q3 ×3,5–3,9

+0,2–0,3 s

mỗi truy vấn của Delta khi bật deletion vectors, kể cả sau khi đã purge hết

RQ3: Metadata và time travel

Metadata tăng tuyến tính; độ trễ time travel đi theo số delete/log file, không theo kích thước metadata



11 KB metadata mỗi batch của Delta CoW

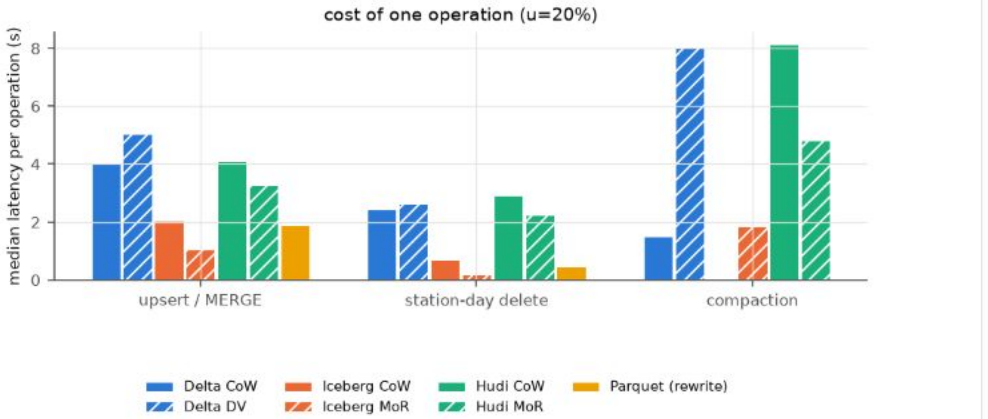
72 KB của Iceberg (Delta DV 67 KB, Hudi 32 KB)

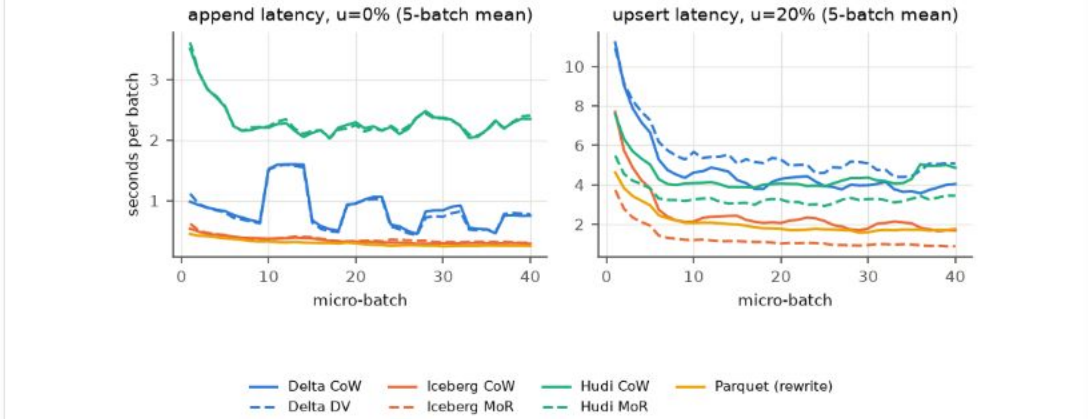
23 phiên bản Hudi còn time travel được: archive dù đã tắt cleaner

Time travel về T−5: Iceberg CoW 0,04–0,12 s, Delta ≈ 1,5 s (phẳng nhờ checkpoint); Iceberg MoR 0,30 → 0,70 s và Hudi MoR 0,68 → 1,22 s khi delete tích lũy

Theo thao tác và theo thời gian

Độ trễ nạp không tăng khi bảng lớn dần; Delta chậm hẳn ở mỗi commit thứ 10





0,19 s
xóa một trạm-ngày trên Iceberg MoR; Delta, Hudi 2,2–2,9 s

5,1 s
commit thứ 10 của Delta (ghi checkpoint), bình thường 0,6 s

7×
append của Hudi so với Iceberg, do small-file handling

Những “bẫy” khi dùng thực tế

Mặc định và trường hợp biên làm sai kết quả hoặc sai dữ liệu nếu không để ý

1 Khóa timestamp của Hudi

bulk_insert và upsert biểu diễn khóa khác nhau → bản ghi trùng im lặng. Checksum phát hiện; dùng khóa chuỗi.

2 Lịch compaction Hudi

Mặc định chỉ sau 5 delta commit: lệnh compaction có thể không làm gì. Đặt chu kỳ tường minh.

3 Archive của Hudi

Chỉ ≈ 23 instant còn time travel: commit vài phút một lần → lùi được 1–2 giờ. Nâng cửa sổ archive.

4 Delete file “treo”

rewrite_data_files của Iceberg 1.6 để lại 23 delete file hết tác dụng. Dọn riêng sau compaction.

5 Checkpoint đồng bộ

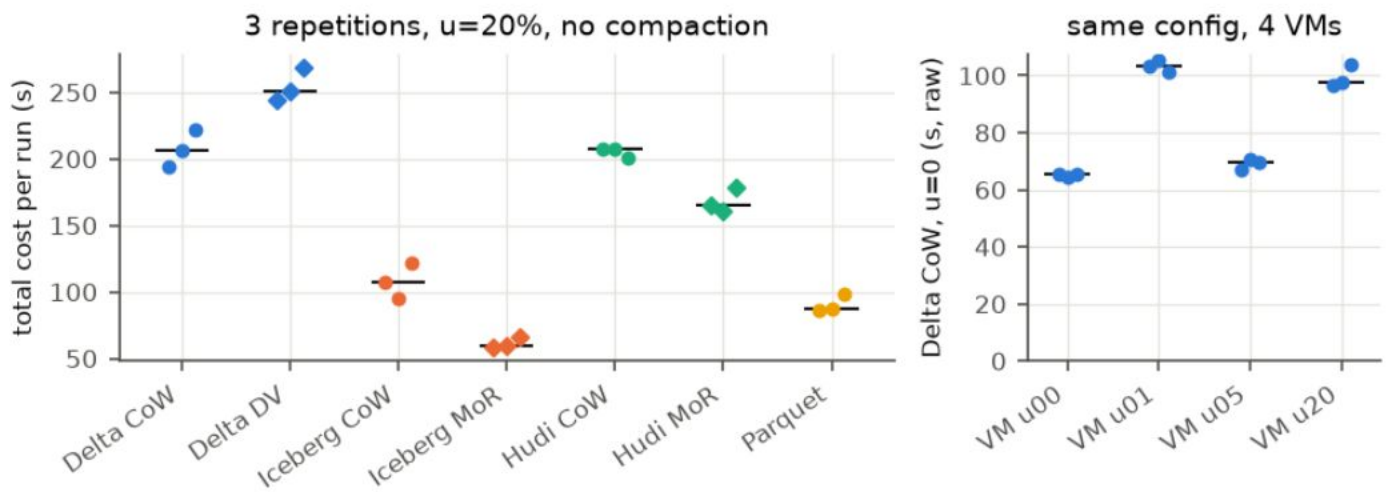
Mỗi commit thứ 10 của Delta chậm gấp nhiều lần. Tính vào ngân sách độ trễ nạp.

6 Mặc định nén khác nhau

Iceberg zstd, Hudi gzip, Delta snappy. Thống nhất bộ nén khi so sánh.

Độ biến thiên và hiệu chuẩn

Lặp trên cùng VM rất ổn định; giữa các VM cùng loại chênh tới 1,6 lần



3,5%
CV median qua 3 lần lặp (tối đa 12,7%)

65 → 103 s
cùng cấu hình trên các VM khác nhau

237 / 237
lần chạy khớp ground truth

Hạn chế và rủi ro

Mỗi hạn chế kèm hướng giảm thiểu

1 Một node, đĩa cục bộ

Chưa có object storage, nơi mỗi file là một request. **Giảm thiểu:** cụm 3–4 node với MinIO.

2 VM dùng chung

Tốc độ VM chênh tới 1,58 lần. **Giảm thiểu:** lặp 3 lần, xen kẽ, hiệu chuẩn, so format trong cùng VM.

3 Quy mô dữ liệu nhỏ

≈ 222 MB, chưa chạy TPC-DS SF10–100. **Giảm thiểu:** nêu rõ phạm vi; thêm TPC-DS SF10.

4 Giá trị d1 gần ngẫu nhiên

Nén kém hơn dữ liệu trạm thật. **Giảm thiểu:** chỉ so sánh tỷ lệ giữa các format.

5 Phụ thuộc phiên bản

Đúng cho phiên bản đã chốt. **Giảm thiểu:** công bố mọi cấu hình trong repo.

6 Một engine

Chỉ Spark. **Giảm thiểu:** đọc đa engine với Trino, DuckDB.

Kết luận và khuyến nghị

Với hệ thống giám sát nhỏ có dữ liệu trễ: Iceberg Merge-on-Read, compaction mỗi 20 batch

RQ1

Có dữ liệu trễ hay không quyết định chi phí: ×1,4–3,8 rồi đứng yên trong 8% từ 1% đến 20%. Iceberg MoR rẻ nhất.

RQ2

Compaction mỗi 20 batch khôi phục độ trễ đọc của CoW với vài giây; mỗi 5 batch chỉ tốn thêm.

RQ3

Metadata tăng 11–72 KB mỗi batch; time travel đi theo số delete/log file. Hudi giữ ≈ 23 phiên bản.

Tình huống	Layout	Compaction
Có dữ liệu trễ (u = 1–20%)	Iceberg MoR	mỗi 20 batch
Chỉ append, cần ACID / lịch sử	Iceberg	mỗi 20 batch
Đã dùng Delta Lake / Hudi	Delta CoW (tắt DV) / Hudi MoR	tùy chọn / mỗi 20

Cảm ơn thầy và các bạn đã lắng nghe

Hà Văn Vinh · Đỗ Thị Lan

Mã nguồn: bộ sinh workload sensor-LST, adapter, notebook Kaggle, dữ liệu đo thô · Demo: Streamlit trên localhost (run_demo.bat)

